Wrocław University of Technology

Lecture 1:
Enterprise applications, SOA

dr inż. Tomasz Walkowiak

My background

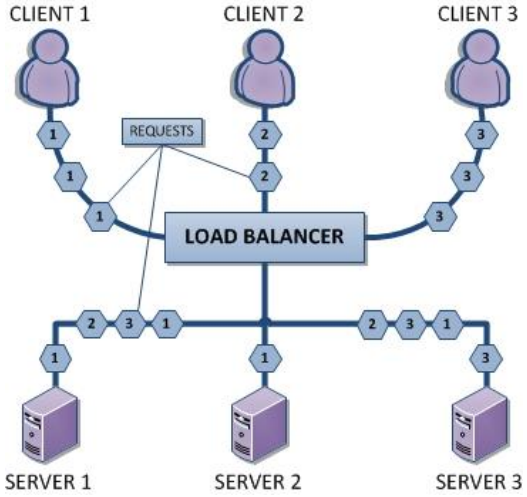
- Languages:
  - C++ => Java => Python
- Deployments:
  - VMware (late 90), Xen (2013)
  - Docker (2016), Kubernetes (2018)
  - RabbitMQ (2014), NATS (2017)
- Projects:
  - DESEREC,
  - CLARIN-PL (CTJ), CLARIN-PL-BIZ, PLLuM https://pllum.clarin-pl.eu

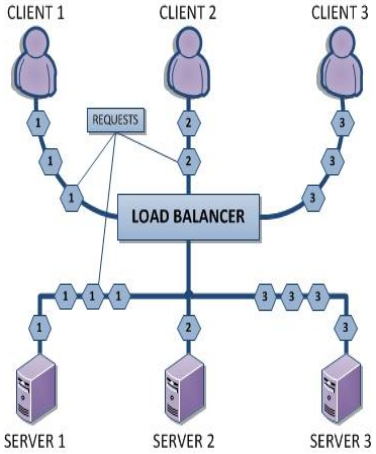
Effective application

- sequential programming
  - effective algorithms (less than n^2)
  - effective data structures (maps)
- multithreading
  - aims: (i) deals with blocking code (i/o) and (ii) parallel processing
  - problems:
    - synchronization (students), costs of creation (partial solution: pools)
    - python – only one core, it is solved now
- processes
  - isolated – primitive communication, costs of creation (pools)
- asynchronous programming
  - mimics event processing
  - solves (i), for (ii) requires processes
  - requires caution during coding (easy to block)
  - one thread

Problems in distributed applications

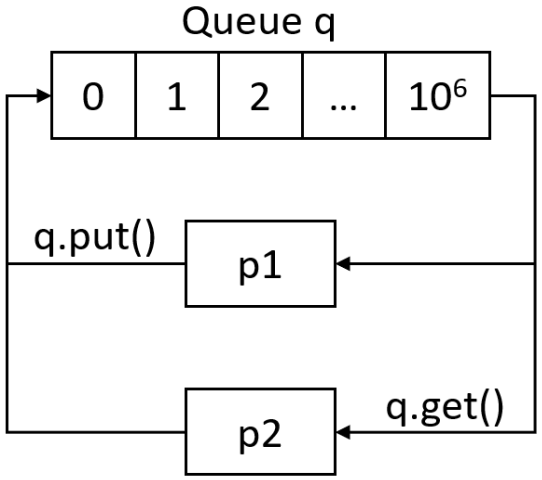
- Bottelnecks
  - databases
  - resource limits
- Partial solutions
  - load balancing
  - stateless applications





Solutions

- Threads/coroutines = share the memory
- Processes
  - almost only communication
  - limit the maximum number
- Synchronization primitives
  - input/output values, but use pools
  - semaphores
  - queues - synchronization/communication (in/out)
  - TCP/IP (even one one host)
- Good design
  - think about future needs
  - optimize only when it is needed
  - pipelines (like in processor) + queues
  - example



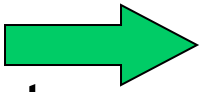
Enterprise Goals

- Reuse 
- Time-to-market ⟹ quicker 
- Cost ⟹ Cheaper 
- Quality ⟹ Better 
- Portability - “write once, deploy anywhere”

Enterprise Applications

Stock Trading System | Banking Application | Network Management System

- Characteristics
- Distributed applications
- Multiple clients
- Multiple servers
- Database(s)



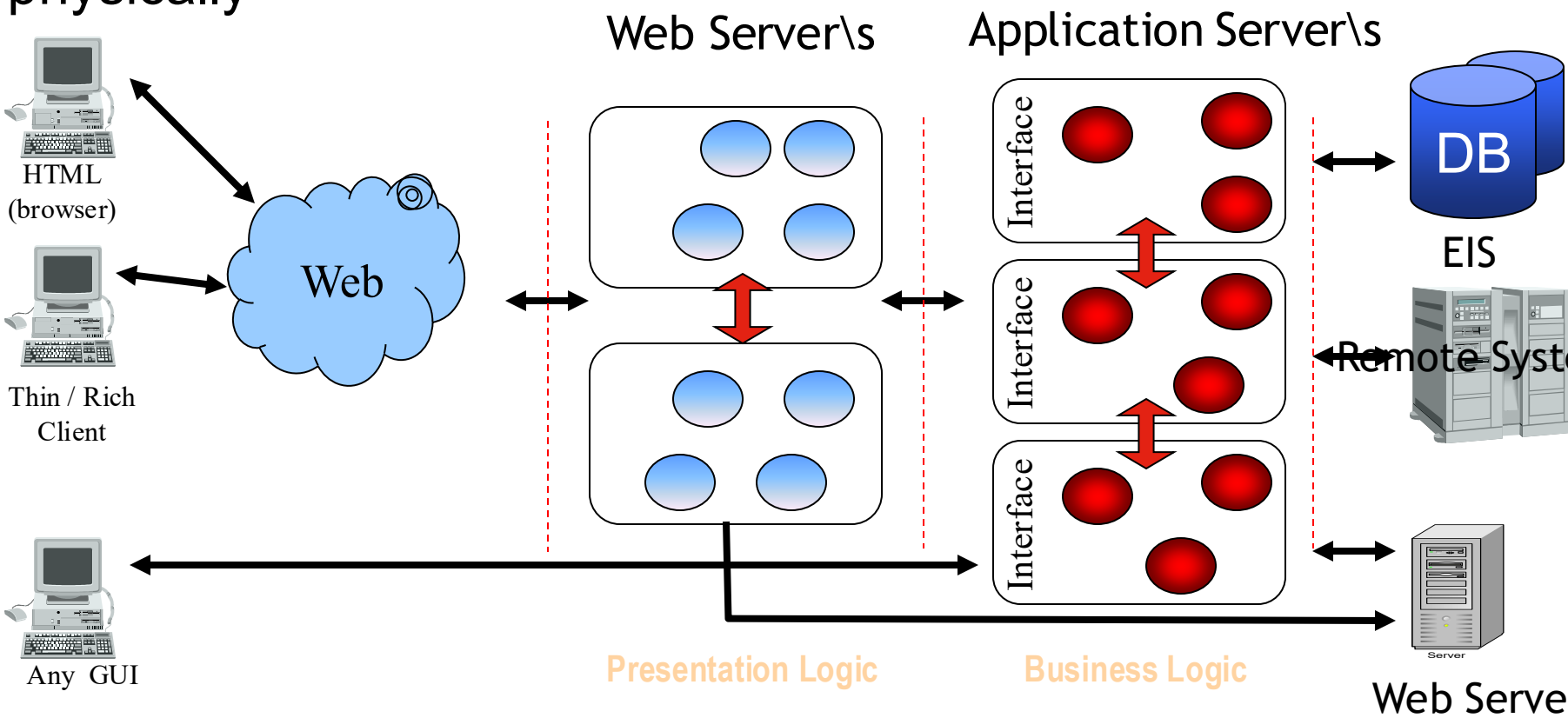
Middleware Services

- Remote Method invocations
- Load balancing
- Transparent fail-over
- Back-end integration
- Transactions
- Clustering
- System Management
- Threading
- Message Oriented Middleware
- Resource pooling
- Security
- Caching

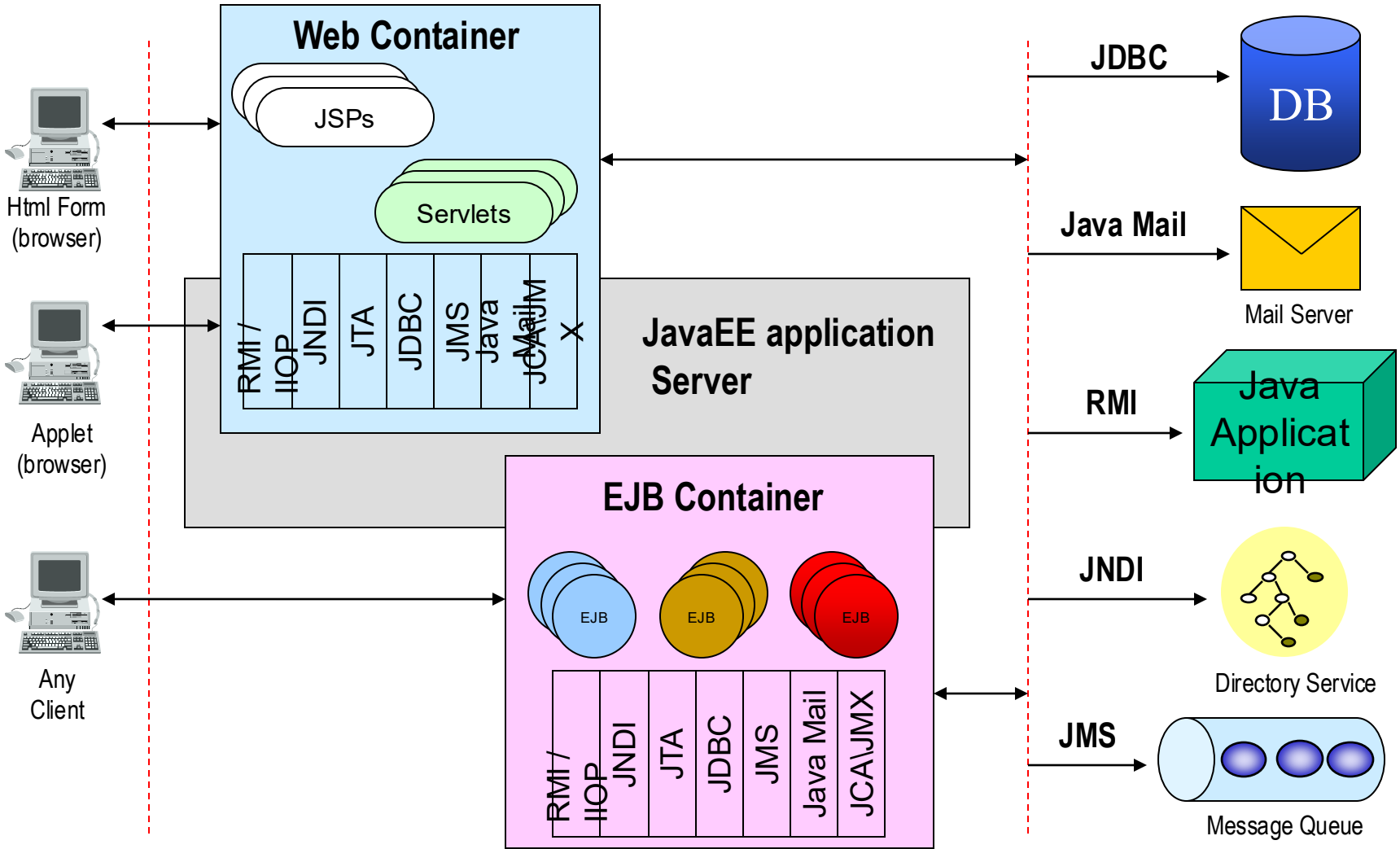
Application Server

N - Tier (Enterprise) Architecture

The application logic is divided by function rather than physically

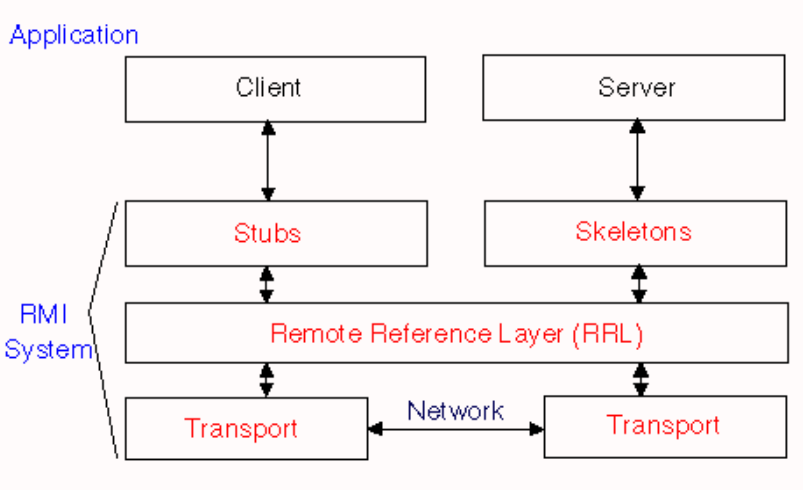


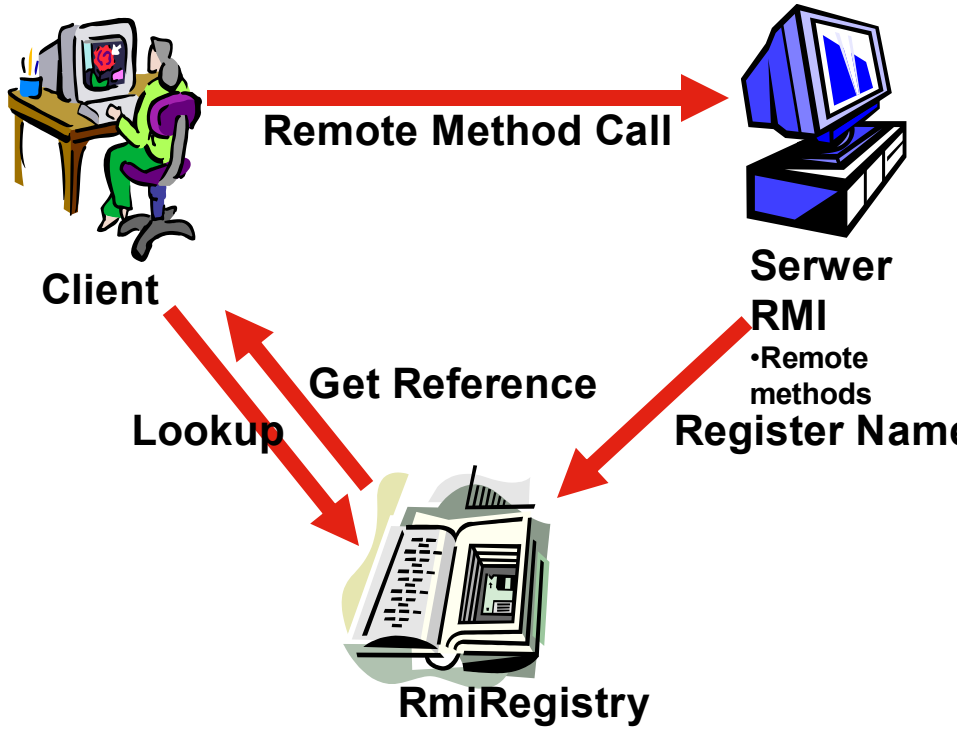
JavaEE Standard Services



Distributed programming

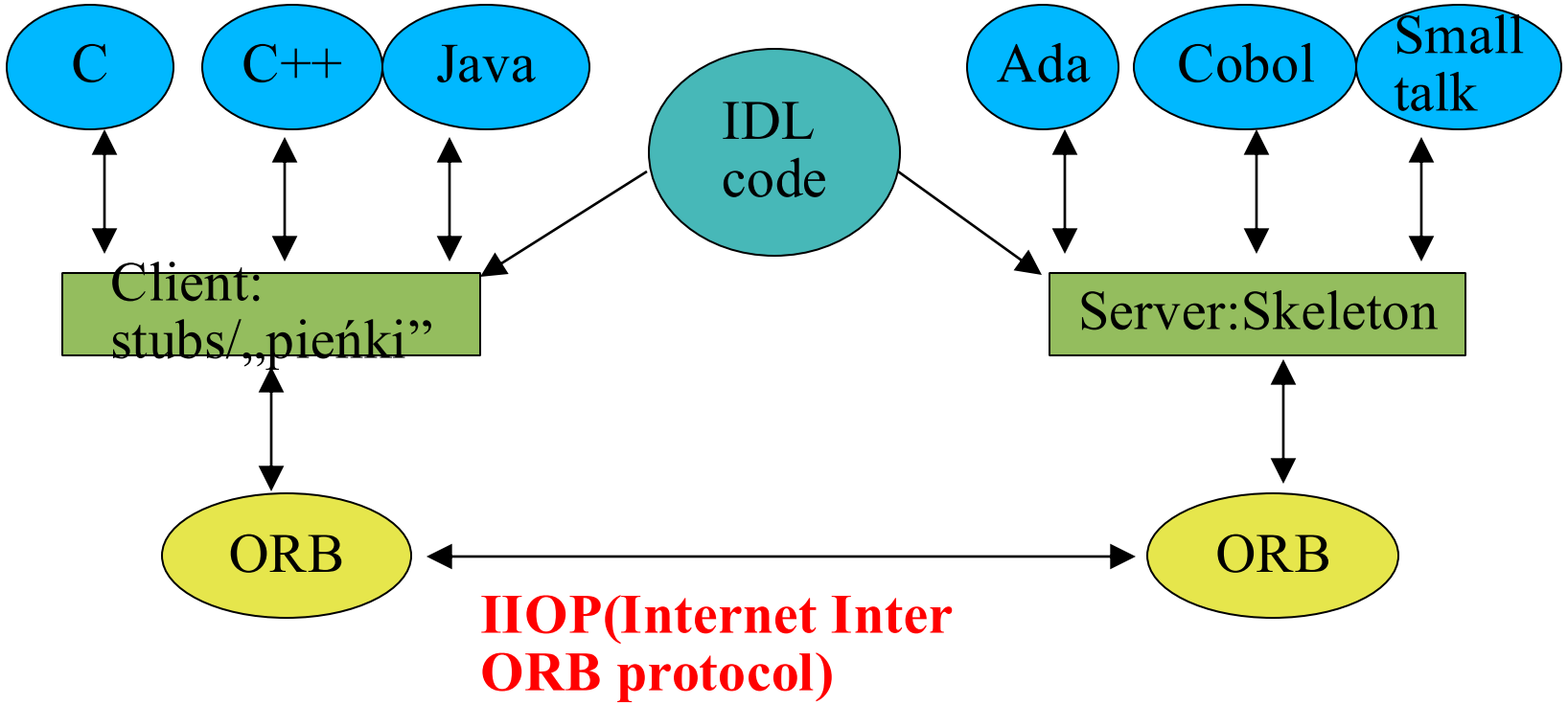
- RMI – remote method invocation
  - Java homogenous





- CORBA
  - heterogenous

Structure


C
C++
Java
IDL code
Ada
Cobol
Small talk
Client: stubs/„pieńki”
Server:Skeleton
ORB
ORB
IIOP(Internet Inter ORB protocol)

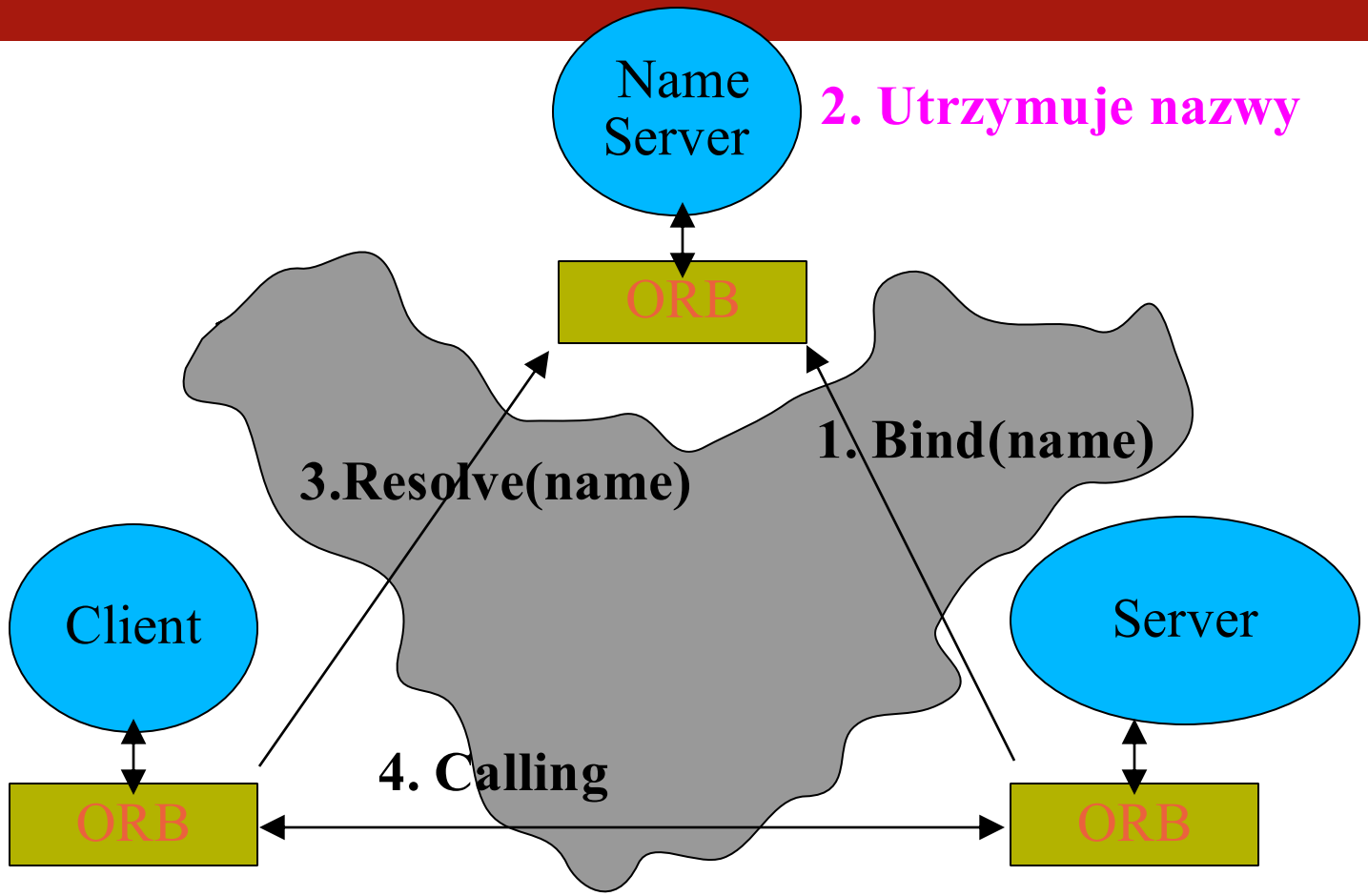
.

Interface Definition Language (IDL)

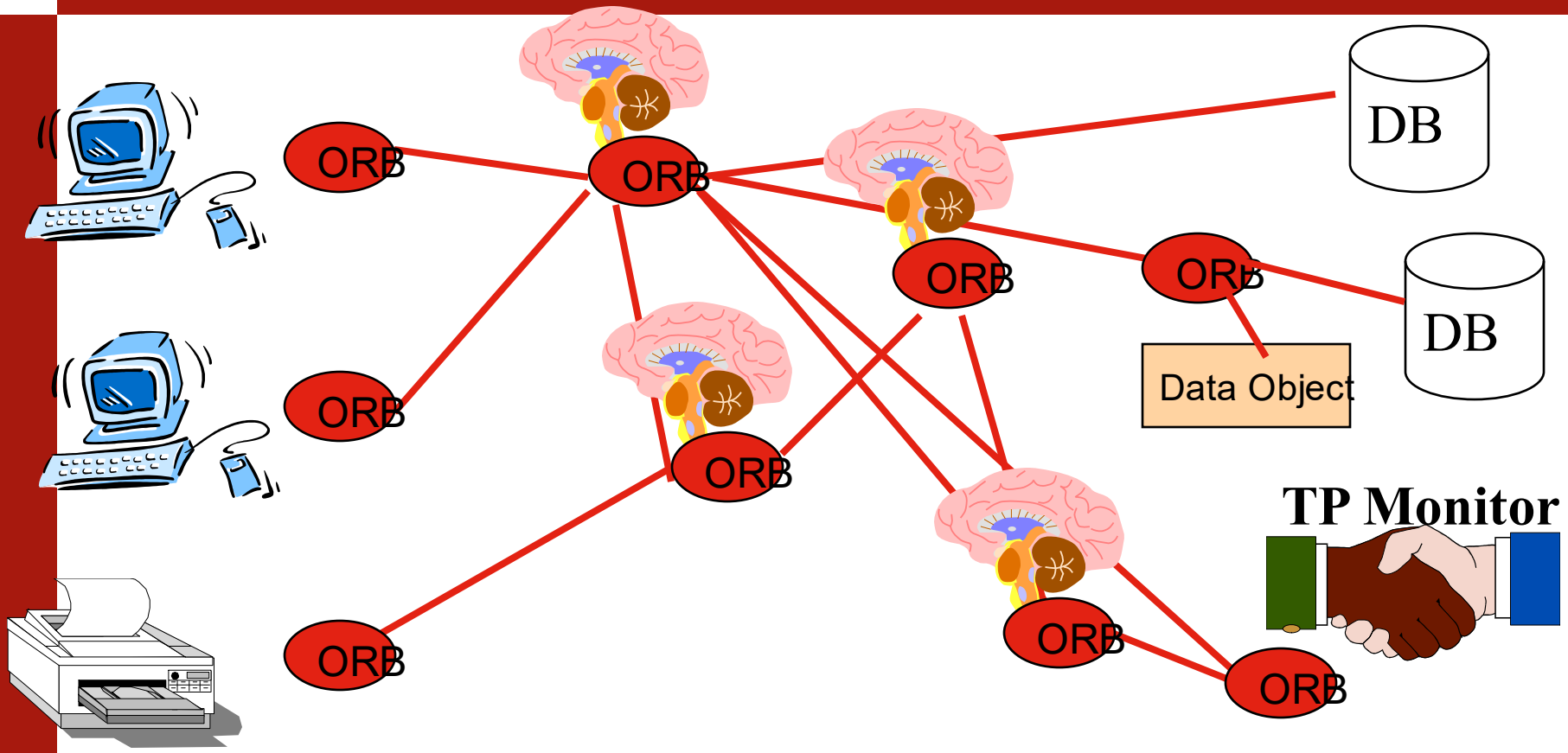
- For interfaces
- Similiar to C++
- Example:

```
module Tutorial {
    module GettingStarted {
        interface Adder {
            double add(in double arg0,in double arg1 );
        };
    };
};
```

Name server


Name Server
2. Utrzymuje nazwy
ORB
1. Bind(name)
3.Resolve(name)
Client
Server
4. Calling
ORB
ORB

N-tiers in CORBA



(by: Orfali et al. „Instatnt Corba")

CORBA problems

- Firewalls
- Security

- What we need:
  - communication protocol
  - data exchange format
  - IDL – interface exchange

SOA Defined

- SOA is a software architecture model
  - in which business functionality are logically grouped and encapsulated into
    - self contained,
    - distinct and reusable units

      called **services that**
    - represent a high level business concept
    - can be distributed over a network
    - can be reused to create new business applications
    - contain contract with specification of the purpose, functionality, interfaces (coarse grained), constraints, usage

      ... of the business functionality

Services are autonomous, discrete and reusable units of business functionality exposing its capabilities in a form of contracts.

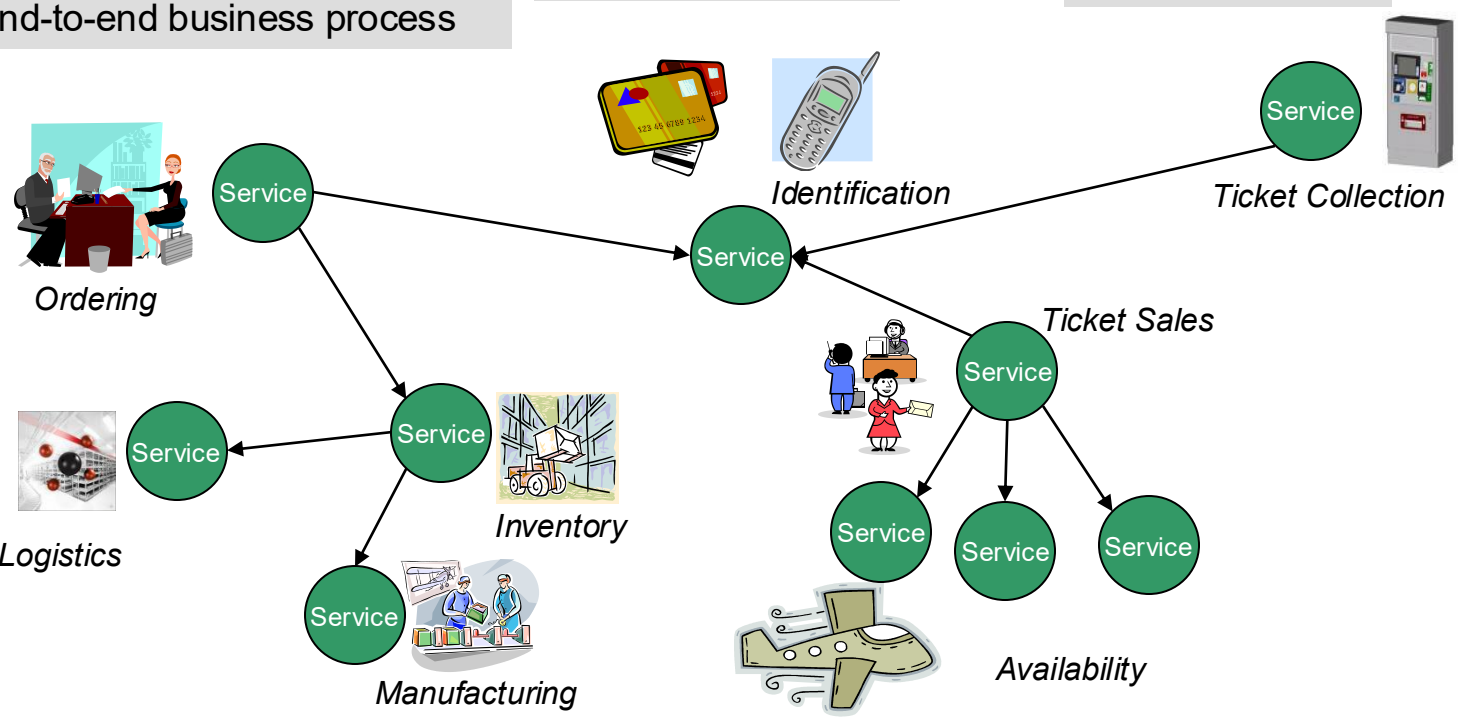
Services can be independently evolved, moved, scaled even in runtime.

Why SOA?
To enable Flexible, Federated Business Processes

Enabling a virtual federation of participants to collaborate in an end-to-end business process

Enabling alternative implementations

Enabling reuse of Services



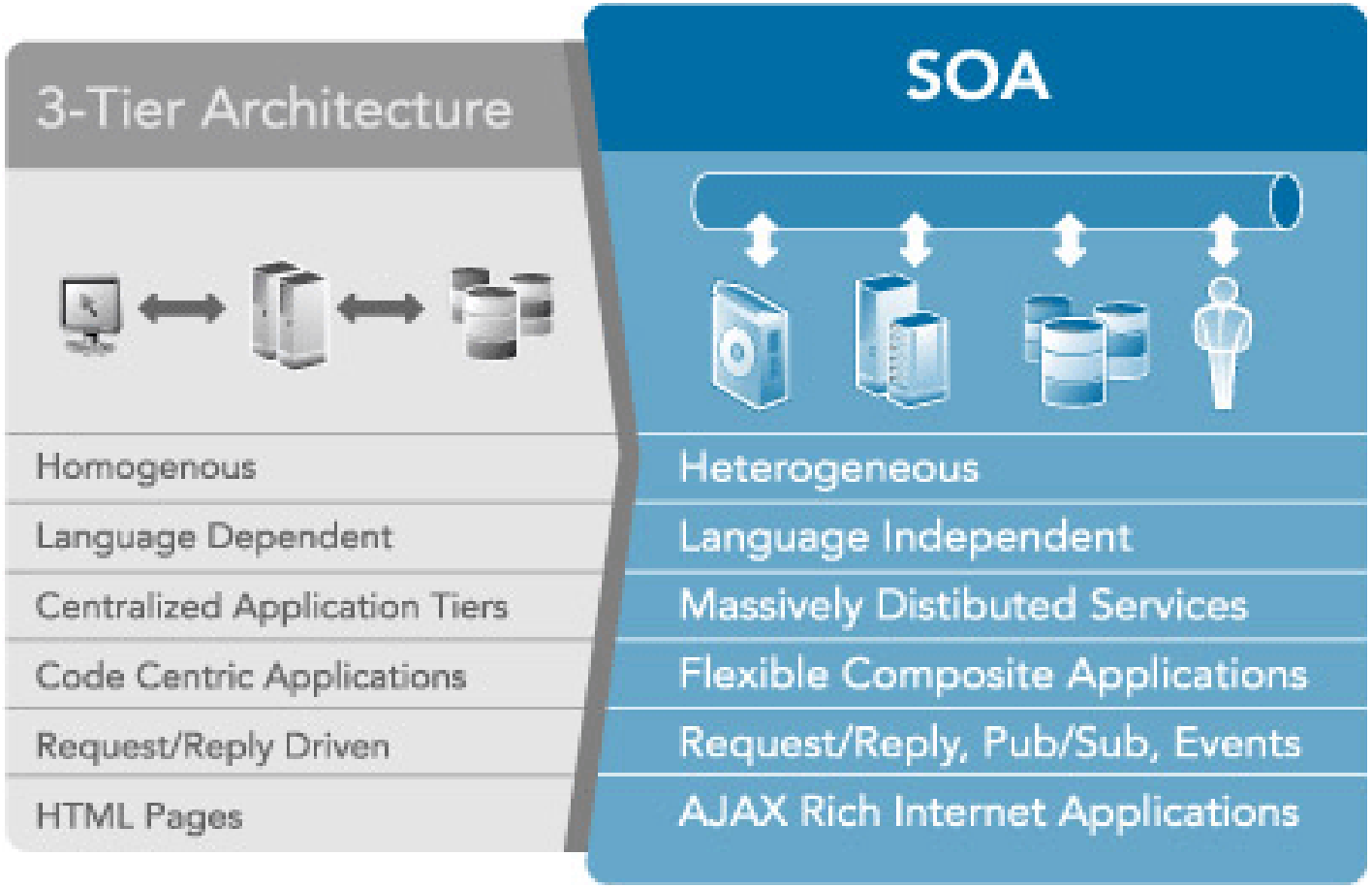
Enabling virtualization of business resources

Enabling aggregation from multiple providers

source:TietoEnator AB.

SOA is an evolutionary step

- for architecture



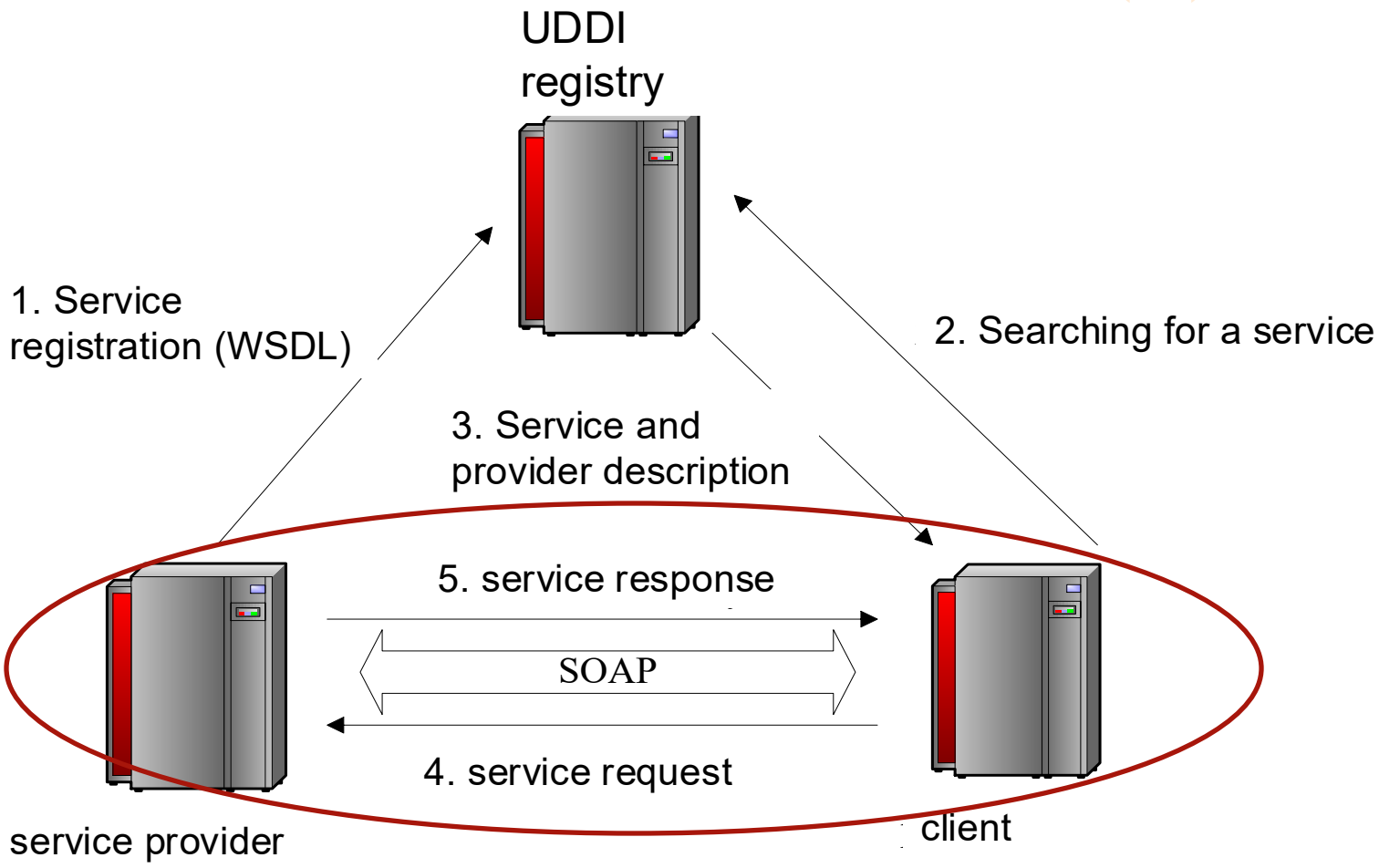
What is a Web service?

- Definition form W3C:
  - a software system identified by a URI
  - public interfaces and bindings are defined and described using XML
  - its definition can be discovered by other software systems
  - other systems may then interact with the Web service in a manner prescribed by its definition, using XML based messages conveyed by Internet protocols
- Intended to support machine-to-machine interactions over the network using messages
- Basic ideas is to build a platform and programming language-independent distributed invocation system out of existing Web standard
- Very loosely defined, when compared to CORBA, RMI, etc
- In comparison to the other RPC (Corba, RMI)
  - no problems with firewalls (HTTP) and less problems with security (HTTPS)
- XML-based distributed services system, with XML based protoclos
  - SOAP, WSDL, UDDI

WebService XML protocols (1)

- UDDI: Universal Description, Discovery and Integration
  - registry of business web services
  - allows to find needed web service on then contact with them
- SOAP: Simple Object Access Protocol
  - XML Message format between client and service
- WSDL: Web Service Description Language
  - describes how the service is to be used
  - like java Interface.
  - guideline for constructing SOAP messages.
  - WSDL is an XML language for writing APIs
- UDDI is now in a rare use
  - IBM, Microsoft, and SAP closed their public UDDI nodes in 2006
- In practice one doesn't need to work directly with SAOP and WSDL
  - there are many tools that generate the WSDL for you from class
  - SOAP messages are constructed automaticaly

WebService XML protocols (2)



SOAP request example

```
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope xmlns:SOAP-
   ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<SOAP-ENV:Body>
<ns1:add xmlns:ns1="urn:Calculator"
SOAP-
   ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<a xsi:type="xsd:float">2.5</a>
<b xsi:type="xsd:float">3.5</b>
</ns1:add>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

SOAP response example

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8
Content-Length: 446
Date: Sun, 15 Dec 2010 23:15:58 GMT
Server: Apache Tomcat/6 (HTTP/1.1 Connector)
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope xmlns:SOAP-
   ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<SOAP-ENV:Body>
<ns1:addResponse xmlns:ns1="urn:Calculator"
SOAP-
   ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<return xsi:type="xsd:float">6.0</return>
</ns1:addResponse>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

SOAP

- **S**imple **O**bject **A**ccess **P**rotocol
- Format for sending messages over Internet between programs
- XML-based
- Platform and language independent
- Simple and extensible
- Stateless, one-way
  - But applications can create more complex interaction patterns

SOAP Building Blocks

- Envelope (required) – identifies XML document as SOAP message
- Header (optional) – contains header information
- Body (required) –call and response information
- Fault (optional) – errors that occurred while processing message

Simple Example

"Get the price of apples"

```
<?xml version="1.0"?>
<soap:Envelope
   xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
   soap:encodingStyle="http://www.w3.org/2001/12/soap-
   encoding">
<soap:Body>
     <m:GetPrice
   xmlns:m="http://www.w3schools.com/prices">
               <m:Item>Apples</m:Item>
     </m:GetPrice>
</soap:Body>
</soap:Envelope>
```

WSDL

```
<xs:complexType name="MathInput">
            <xs:sequence>
               <xs:element name="x" type="xs:double"/>
               <xs:element name="y" type="xs:double"/>
            </xs:sequence> ...
<xs:element name="Add" type="MathInput"/>
<xs:element name="AddResponse" type="MathOutput"/> …
<message name="AddMessage">
      <part name="parameters" element="ns:Add"/>
</message> …
<portType name="MathInterface">
<operation name="Add">
         <input message="y:AddMessage"/>
         <output message="y:AddResponseMessage"/>
</operation>
```

Use of WSDL Documents

- Description of service interfaces:
  - Compile-time:
    - Developer uses WSDL before service deployment.
  - Run-time:
    - Client downloads WSDL description and uses the information it provides to execute the service.
- As a side-effect:
  - Developers can use WSDL to speed up the development of code,
  - WSDL→Java code,
  - Java interfaces → WSDL.

WebService conclusion

- Advantages
  - very simple to deploy
    - server and client
  - no problems with firewall (HTTP)
  - security (HTTPS)
  - easy way of communication in the heterogonous environment
    - similar approach in .Net
- Disadvantages
  - large size of messages (SOAP in XML)
  - “heavy” servers
    - Not true today (for example JAX-WS and Java SE)